Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 5

The File System with Your Own Hands

Track A: Warm-up on Linux

Elaborated by:
st. gr. FAF-243
Veacheslav Morari

Verified by:
Patricia Reitman

Chișinău – 2026

Contents

Introduction **2**

1. The tree and the file types **2**

1.1. Observations 4

2. Inodes and links **5**

2.1. An inode and a name 5

2.2. A hard link and removing the original 5

2.3. A symbolic link and removing its target 5

2.4. Link counts of directories 6

2.5. Size is not space 6

2.6. Observations 6

3. Permissions **7**

3.1. Files and umask 7

3.2. Directories 7

3.3. Special bits and other people's files 8

3.4. Observations 8

4. Disks and the kernel as files **9**

4.1. Observations 10

Conclusion **10**

Introduction

The purpose of this laboratory work is to check on a real Linux system what was shown in the lecture about the file system: the directory tree and the types of files, inodes, hard and symbolic links, permissions with the special bits, and the disks and the kernel interface `/proc`. No programs are written; only standard commands are used, and the results are explained.

The work was done on Ubuntu 26.04.1 LTS in a virtual machine, as the normal user `veacheslav` and not as `root`, because `root` ignores most permission checks and the experiments would give wrong results. All files were created in the folder `~/os-lab5`.

1. The tree and the file types

The working folder was created and the root directory was listed. The first letter of the line printed by `ls -l` gives the type of an object: `-` is a regular file, `d` a directory, `l` a symbolic link, `c` a character device and `b` a block device.

```
veacheslav@os-lab:~/os-lab5$  whoami
veacheslav
veacheslav@os-lab:~/os-lab5$  mkdir -p ~/os-lab5
veacheslav@os-lab:~/os-lab5$  cd ~/os-lab5
veacheslav@os-lab:~/os-lab5$  pwd
/home/veacheslav/os-lab5
veacheslav@os-lab:~/os-lab5$  ls /
bin   cdrom  etc   lib    lost+found  mnt  proc  run   snap  sys  usr
boot  dev    home  lib64  media       opt  root  sbin  srv   tmp  var
```

Figure 1.1 – The user, the working folder and the root directory

```
veacheslav@os-lab:~/os-lab5$  ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 20 08:46 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct 19 11:34 /dev/null
drwxr-xr-x 142 root root 12288 Oct 17 13:41 /etc
-rw-r--r--   1 root root  2829 Oct 12 12:28 /etc/passwd
drwxrwxrwt  15 root root   360 Oct 19 11:36 /tmp
```

Figure 1.2 – Types of five paths

```
veacheslav@os-lab:~/os-lab5$  ls -l /dev | head
total 0
crw-r--r--   1 root       root     10, 235 Oct 19 11:34 autofs
drwxr-xr-x   2 root       root        520 Oct 19 11:35 block
drwxr-xr-x   2 root       root         80 Oct 19 11:34 bsg
crw-------   1 root       root     10, 234 Oct 19 11:34 btrfs-control
drwxr-xr-x   3 root       root         60 Oct 19 11:34 bus
lrwxrwxrwx+  1 root       root          3 Oct 19 11:34 cdrom -> sr0
drwxr-xr-x   2 root       root       3840 Oct 19 11:34 char
crw-------   1 root       tty       5,   1 Oct 19 11:34 console
lrwxrwxrwx   1 root       root         11 Oct 19 11:34 core -> /proc/kcore
veacheslav@os-lab:~/os-lab5$  file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
veacheslav@os-lab:~/os-lab5$  du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
459M    /var/log
5.1G    /usr
```

Figure 1.3 – The beginning of /dev, the file command and the sizes of /var/log and /usr

```
veacheslav@os-lab:~/os-lab5$  ls -l /dev | grep '^b' | head
brw-rw----   1 root       disk      7,   0 Oct 19 11:34 loop0
brw-rw----   1 root       disk      7,   1 Oct 19 11:34 loop1
brw-rw----   1 root       disk      7,  10 Oct 19 11:34 loop10
brw-rw----   1 root       disk      7,  11 Oct 19 11:34 loop11
brw-rw----   1 root       disk      7,  12 Oct 19 11:34 loop12
brw-rw----   1 root       disk      7,  13 Oct 19 11:34 loop13
brw-rw----   1 root       disk      7,  14 Oct 19 11:34 loop14
brw-rw----   1 root       disk      7,  15 Oct 19 11:34 loop15
brw-rw----   1 root       disk      7,  16 Oct 19 11:34 loop16
brw-rw----   1 root       disk      7,  17 Oct 19 11:34 loop17
```

Figure 1.4 – Block devices in /dev

```
veacheslav@os-lab:~/os-lab5$  lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
loop1    7:1    0  66.8M  1 loop /snap/core24/1643
loop2    7:2    0    20M  1 loop /snap/desktop-security-center/151
loop3    7:3    0  66.8M  1 loop /snap/core24/2124
loop4    7:4    0 260.3M  1 loop /snap/firefox/8763
loop5    7:5    0  14.2M  1 loop /snap/desktop-security-center/206
loop6    7:6    0  16.5M  1 loop /snap/firmware-updater/226
loop7    7:7    0  12.7M  1 loop /snap/firmware-updater/262
loop8    7:8    0 614.5M  1 loop /snap/gnome-46-2404/164
loop9    7:9    0   1.5M  1 loop /snap/hwctl/123
loop10   7:10   0  91.7M  1 loop /snap/gtk-common-themes/1535
loop11   7:11   0  44.7M  1 loop /snap/snapd/28254
loop12   7:12   0  18.8M  1 loop /snap/prompting-client/222
loop13   7:13   0   402M  1 loop /snap/mesa-2404/1839
loop14   7:14   0  18.4M  1 loop /snap/prompting-client/228
loop15   7:15   0  11.8M  1 loop /snap/snap-store/1390
loop16   7:16   0  50.1M  1 loop /snap/snapd/27710
loop17   7:17   0  44.7M  1 loop /snap/snapd/28254
loop18   7:18   0   828K  1 loop /snap/snapd-desktop-integration/391
sda      8:0    0  50.4G  0 disk
├─sda1   8:1    0     1M  0 part
└─sda2   8:2    0  50.4G  0 part /
sr0     11:0    1  50.7M  0 rom  /run/media/veacheslav/VBox_GAs_7.2.20
```

Figure 1.5 – The disks and partitions shown by lsblk

1.1. Observations

(1) The path `/etc` is a directory (`d`), `/etc/passwd` is a regular file (`-`), `/bin` is a symbolic link (`l`), `/dev/null` is a character device (`c`) and `/tmp` is a directory (`d`) whose last permission letter is `t`, the sticky bit.

(2) On this system `/bin` is not a directory but a symbolic link to `usr/bin`. The command `file` confirms it. It also shows that `/usr/bin/ls` is itself a link, to `../lib/cargo/bin/coreutils/ls`, and that `/etc/passwd` is plain ASCII text.

(3) The device `/dev/null` is a character device (`c`, major 1, minor 3), and `/dev/loop0` is a block device (`b`, major 7, minor 0). Block devices are the ones that are read and written in blocks, such as disks and the loop devices used here for Snap packages. The listing of `/dev` also contains directories and links, for example `cdrom -> sr0`.

(4) The folder `/usr` is bigger: it takes 5.1 GB, while `/var/log` takes 459 MB, so `/usr` is about 4.6 GB larger. This is expected, because `/usr` holds the programs and libraries of the system and `/var/log` holds only logs.

2. Inodes and links

2.1. An inode and a name

A file `notes.txt` with the text `hello` was created and examined with `stat` and `ls -li`. The size is 6 bytes (five letters and the newline), the inode number is 1701749 and the file has one link.

```
veacheslav@os-lab:~/os-lab5$  echo hello > notes.txt
veacheslav@os-lab:~/os-lab5$  stat notes.txt
  File: notes.txt
  Size: 6          Blocks: 8          IO Block: 4096   regular file
Device: 8,2  Inode: 1701749     Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/veacheslav)   Gid: ( 1000/veacheslav)
Access: 2026-10-19 11:38:40.482874466 +0000
Modify: 2026-10-19 11:38:39.808992893 +0000
Change: 2026-10-19 11:38:39.808992893 +0000
 Birth: 2026-10-19 11:38:39.808992893 +0000
veacheslav@os-lab:~/os-lab5$  ls -li notes.txt
1701749 -rw-rw-r-- 1 veacheslav veacheslav 6 Oct 19 11:38 notes.txt
```

Figure 2.1 – stat and ls -li for notes.txt

2.2. A hard link and removing the original

The command `ln notes.txt hard.txt` created a second name for the same inode: both names show inode 1701749 and the link count became 2. After `rm notes.txt` the file was still readable under the name `hard.txt`, and the link count went back to 1.

```
veacheslav@os-lab:~/os-lab5$  ln notes.txt hard.txt
veacheslav@os-lab:~/os-lab5$  ls -li notes.txt hard.txt
1701749 -rw-rw-r-- 2 veacheslav veacheslav 6 Oct 19 11:38 hard.txt
1701749 -rw-rw-r-- 2 veacheslav veacheslav 6 Oct 19 11:38 notes.txt
veacheslav@os-lab:~/os-lab5$  rm notes.txt
veacheslav@os-lab:~/os-lab5$  cat hard.txt ; ls -li hard.txt
hello
1701749 -rw-rw-r-- 1 veacheslav veacheslav 6 Oct 19 11:38 hard.txt
```

Figure 2.2 – A hard link and the removal of the first name

2.3. A symbolic link and removing its target

The symbolic link `soft.txt` got its own inode, 1701750, and its type is `l`. Reading through it worked while `hard.txt` existed. After `rm hard.txt` the link remained in the directory, but `cat soft.txt` failed with “No such file or directory”.

```
veacheslav@os-lab:~/os-lab5$  ln -s hard.txt soft.txt
veacheslav@os-lab:~/os-lab5$  ls -li hard.txt soft.txt
1701749 -rw-rw-r-- 1 veacheslav veacheslav 6 Oct 19 11:38 hard.txt
1701750 lrwxrwxrwx 1 veacheslav veacheslav 8 Oct 19 11:40 soft.txt -> hard.txt
veacheslav@os-lab:~/os-lab5$  cat soft.txt
hello
veacheslav@os-lab:~/os-lab5$  rm hard.txt
veacheslav@os-lab:~/os-lab5$  cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 veacheslav veacheslav 8 Oct 19 11:40 soft.txt -> hard.txt
```

Figure 2.3 – A symbolic link and a dangling link

2.4. Link counts of directories

A new directory `d` had 2 links. After the creation of `d/x` and `d/y` the count became 4.

```
veacheslav@os-lab:~/os-lab5$  mkdir d ; ls -ld d
drwxrwxr-x 2 veacheslav veacheslav 4096 Oct 19 11:42 d
veacheslav@os-lab:~/os-lab5$  mkdir d/x d/y ; ls -ld d
drwxrwxr-x 4 veacheslav veacheslav 4096 Oct 19 11:42 d
```

Figure 2.4 – The link count of a directory

2.5. Size is not space

A file with 2 bytes occupies 4.0K according to `du`, and a sparse file of 1 GiB created with `truncate` occupies 0. The big file was removed afterwards.

```
veacheslav@os-lab:~/os-lab5$  echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-r-- 1 veacheslav veacheslav 2 Oct 19 11:42 one.txt
4.0K    one.txt
veacheslav@os-lab:~/os-lab5$  truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-r-- 1 veacheslav veacheslav 1.0G Oct 19 11:42 big.img
0       big.img
veacheslav@os-lab:~/os-lab5$  rm big.img
```

Figure 2.5 – Logical size and used space

2.6. Observations

(1) The inode number is the first column of `ls -li` and the field `Inode` in the output of `stat`. The number of names of the file is the field `Links` in `stat`, or the number after the permission string in `ls -l`.

(2) After `rm notes.txt` the file could still be read because the name is not part of the file. A name is only an entry in a directory that points to an inode, and the inode with the data had two such entries. The command `rm` removes one entry and lowers the link count; the data are freed only when the count reaches zero and no process keeps the file open.

(3) The command `cat soft.txt` failed because a symbolic link stores only the path of its target, not the content. When `hard.txt` was removed, the path pointed to nothing, so the link became dangling, although the link itself still exists. The size of `soft.txt` is 8 bytes because it holds the text of the path, `hard.txt`, which has exactly 8 characters.

(4) The link count of a new directory is 2 because the directory is referenced from its parent (its own name there) and from its own entry `.`. Each subdirectory contains an entry `..` that points back to the parent, so two subdirectories add two links, and the total is 4. Plain files do not add anything to the count of the parent.

(5) A file system allocates space in blocks, and the smallest block here is 4 KiB, so a file of 2 bytes takes one whole block, and `du` shows 4.0K. The file `big.img` is sparse: `truncate` only set its length to 1 GiB and wrote no data, so no blocks were allocated and `du` shows 0, while `ls` shows the logical size of 1.0G.

3. Permissions

3.1. Files and umask

The file `s.txt` was given mode 600, then 000, and then the read right of the owner was added with `u+r`. In the state 000 even the owner got "Permission denied". The `umask` of the user is 0002, and the new file and directory got modes 664 and 775.

```
veacheslav@os-lab:~/os-lab5$  echo secret > s.txt
veacheslav@os-lab:~/os-lab5$  chmod 600 s.txt ; ls -l s.txt
-rw------- 1 veacheslav veacheslav 7 Oct 19 11:44 s.txt
veacheslav@os-lab:~/os-lab5$  chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
veacheslav@os-lab:~/os-lab5$  chmod u+r s.txt ; cat s.txt
secret
```

Figure 3.1 – Changing the mode of s.txt

```
veacheslav@os-lab:~/os-lab5$  umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0002
-rw-rw-r-- 1 veacheslav veacheslav    0 Oct 19 11:46 new.txt
drwxrwxr-x 2 veacheslav veacheslav 4096 Oct 19 11:46 newdir
```

Figure 3.2 – umask and the modes of new objects

3.2. Directories

In the directory `box` with the file `f` two modes were tested. With read only (`u=r`) the command `ls box` printed the name `f`, but `cat box/f` was denied. With execute only (`u=x`) `ls box` was denied, but `cat box/f` printed `hi`. At the end the mode was restored with `chmod 755 box`.

```
veacheslav@os-lab:~/os-lab5$  mkdir box ; echo hi > box/f
veacheslav@os-lab:~/os-lab5$  chmod u=r,go= box ; ls box ; cat box/f
f
cat: box/f: Permission denied
veacheslav@os-lab:~/os-lab5$  chmod u=x,go= box ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
veacheslav@os-lab:~/os-lab5$  chmod 755 box
```

Figure 3.3 – Read only and execute only on a directory

3.3. Special bits and other people's files

The program `/usr/bin/passwd` has the letter `s` in the owner's execute position, and `/tmp` has the letter `t` at the end. The file `/etc/shadow` has mode 640, belongs to `root` and to the group `shadow`, so the ordinary user cannot read it.

```
veacheslav@os-lab:~/os-lab5$  ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 93640 Feb  2  2026 /usr/bin/passwd
drwxrwxrwt 15 root root 360 Oct 19 11:42 /tmp
veacheslav@os-lab:~/os-lab5$  cat /etc/shadow
cat: /etc/shadow: Permission denied
veacheslav@os-lab:~/os-lab5$  ls -l /etc/shadow
-rw-r----- 1 root shadow 1338 Oct 12 12:28 /etc/shadow
```

Figure 3.4 – Special bits and /etc/shadow

3.4. Observations

(1) The mode 600 is `rw-------`: a private file, for example a secret key or a password file. The mode 644 is `rw-r--r--`: an ordinary file that everyone may read, such as a document or a web page. The mode 755 is `rwxr-xr-x`: a program or a public directory. The mode 700 is `rwx------`: a private directory of the owner, for example `~/.ssh`.

(2) Ownership does not help because the kernel checks the permission bits of the class that matches the user. For the owner these are the first three bits, and with mode 000 they are all empty, so reading is refused. The owner is only allowed to change the mode with `chmod`, which is what restored access in the experiment.

(3) The `umask` is 0002. New files start from 666 and new directories from 777, and the bits of the mask are removed. The mask 002 removes the write bit for others, so `new.txt` got 664 (`rw-rw-r--`) and `newdir` got 775 (`rwxrwxr-x`), as the listing shows.

(4) On a directory, the right `r` allows reading the list of names, which is why `ls box` worked in the first test and failed in the second. The right `x` allows passing through the directory and reaching an entry whose name is already known, which is why `cat box/f` failed without `x` and worked with it. Without `x` the names in the directory are visible but useless; without `r` the file is reachable only if you know its name.

(5) The letter `s` in `/usr/bin/passwd` is the setuid bit: when any user runs the program, it works with the rights of the owner of the file, here `root`. It is needed because

`passwd` must write to protected files such as `/etc/shadow`, which an ordinary user cannot even read, as the failed `cat /etc/shadow` showed. The letter `t` in `/tmp` is the sticky bit: everybody can create files in this shared directory, but a user can delete or rename only his own files there.

4. Disks and the kernel as files

The disks were listed with `lsblk` (Figure 1.5), and the root file system was examined with `df -h` and `mount`. The kernel information was read from the virtual files `/proc/cpuinfo`, `/proc/meminfo` and `/proc/self/status`, and the open files of a process from `/proc/self/fd`.

```
veacheslav@os-lab:~/os-lab5$  df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        50G  8.3G   39G  18% /
veacheslav@os-lab:~/os-lab5$  mount | grep " / "
/dev/sda2 on / type ext4 (rw,relatime)
```

Figure 4.1 – The root file system according to df and mount

```
veacheslav@os-lab:~/os-lab5$  grep -c processor /proc/cpuinfo
7
veacheslav@os-lab:~/os-lab5$  head -3 /proc/meminfo
MemTotal:        7601204 kB
MemFree:         4187704 kB
MemAvailable:    6148248 kB
veacheslav@os-lab:~/os-lab5$  head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   6562
Ngid:   0
```

Figure 4.2 – CPU count, memory and the status of the current process

```
veacheslav@os-lab:~/os-lab5$  ls -l /proc/self/fd
total 0
lrwx------ 1 veacheslav veacheslav 64 Oct 19 12:17 0 -> /dev/pts/0
lrwx------ 1 veacheslav veacheslav 64 Oct 19 12:17 1 -> /dev/pts/0
lrwx------ 1 veacheslav veacheslav 64 Oct 19 12:17 2 -> /dev/pts/0
lr-x------ 1 veacheslav veacheslav 64 Oct 19 12:17 3 -> /proc/6563/fd
```

Figure 4.3 – The open files of the process that ran ls

4.1. Observations

(1) The disk is `sda` with a size of 50.4 GiB. It has two partitions: `sda1` of 1 MiB, which is not mounted (it is typically a small boot area), and `sda2` of 50.4 GiB, which is mounted at `/`. The file system on `/dev/sda2` is `ext4`, mounted read-write (`rw,relatime`). The loop devices `loop0`–`loop18` are not real disks: they are Snap packages mounted from files, and `sr0` is the virtual CD with the VirtualBox Guest Additions.

(2) The root file system is 18% full: 8.3 GB are used and 39 GB are available out of 50 GB.

(3) The `grep -c processor /proc/cpuinfo` command printed 7, so the virtual machine has 7 CPU cores. The field `MemTotal` is 7 601 204 kB, which is about 7.25 GiB (7.6 GB) of RAM; at that moment 6 148 248 kB were available.

(4) The file `/proc/self/status` showed the name `head`. The path `self` always means the process that opens the file, and the process that read it was `head`, which started in the same command line and read the file itself. The same happens in the last experiment: the link `3 -> /proc/6563/fd` belongs to `ls`, which had the directory open while it listed it, and descriptors 0, 1 and 2 are the terminal `/dev/pts/0`. None of these files is stored on the disk: `/proc` is generated by the kernel when it is read.

Conclusion

In this work the main parts of the Linux file system were checked by experiment. The tree contains regular files, directories, links and device files, and the first letter of `ls -l` tells them apart. A name is only an entry in a directory that points to an inode, so several hard links can share one file, removing one name does not remove the data, and a symbolic link, which keeps only a path, breaks when its target disappears. The link counts of directories follow from the entries `.` and `..`, and the difference between the logical size and the used space is explained by block allocation and sparse files. The permission experiments showed that the owner is still bound by the mode bits, that `r` and `x` mean different things on a directory, that `umask` decides the modes of new objects, and that the setuid and sticky bits solve the problems of changing a password and of sharing `/tmp`. The last part showed that the disk is split into partitions with an `ext4` file system mounted at `/`, and that the kernel exposes its own data, such as the number of CPUs, the memory size and the state of a process, as ordinary files in `/proc`.